SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**
Question Count: 5

Code of Conduct

I Dinnepati Sindhu Prasad-192311271 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dinnepati Sindhu Prasad

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed

messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Website, Mobile Interaction and Customer Transaction Platform

The described platform handles billions of clicks, mobile interactions, and transactions every day. From the scale and type of data, it is likely designed using distributed storage, parallel processing, and real-time streaming technologies.

Probable Storage Architecture

A distributed file system such as **Hadoop HDFS** or cloud object storage such as Amazon S3 is likely used as the main storage layer. It can store very large volumes of data across multiple machines.

The platform would probably maintain two types of data:

- **Structured data:** Customer details, payments, transactions, and product information can be stored in relational databases or data warehouses.
- **Semi-structured data:** Website clicks, application events, JSON logs, and browsing information can be stored in a data lake.

Partitioning data by **date, region, customer segment, or event type** would make queries faster and reduce the amount of data that needs to be processed.

Data Ingestion

An event streaming platform such as **Apache Kafka** is likely used to collect website and mobile events. Batch data can be imported using ETL tools or systems such as Apache NiFi.

Kafka topics can separate different types of events, such as:

Clicks → Mobile Events → Transactions → Customer Activity

Batch Processing

For large historical datasets, **Apache Spark** or Hadoop MapReduce is likely used. Spark can process large datasets in parallel and perform operations such as customer behavior analysis, sales analysis, and trend identification.

Stream Processing

Real-time analytics can be implemented using **Apache Flink, Spark Structured Streaming, or Kafka Streams**.

For example, when thousands of users click on a product at the same time, the streaming system can immediately calculate the popularity of that product.

Scalability and Partitioning

The platform would probably use horizontal scaling, where additional servers can be added when the workload increases.

Kafka partitions allow events to be processed in parallel. Storage can also be partitioned by date and region. Replication would be used so that failure of one server does not cause data loss.

Analytics and Dashboards

After processing, summarized data can be stored in a data warehouse or analytical database. Business intelligence tools such as **Power BI, Tableau, or Looker** can connect to this layer.

The dashboards may display:

- Daily website traffic
- Customer activity
- Sales performance
- Popular products
- Regional trends
- Conversion rates

Probable Architecture

Website / Mobile Apps / Transactions
↓
Kafka / Data Ingestion
↓
Data Lake + Structured Database
↓
Spark / Flink Processing
↓
Analytics Warehouse
↓
Power BI / Tableau / Business Dashboards

2. E-Commerce Personalization Platform

The platform tracks product views, cart additions, abandoned checkouts, and payments. This indicates that it needs both real-time event processing and historical customer analysis.

Event Streaming

Apache Kafka is a likely choice for collecting customer events.

Different Kafka topics could contain:

- Product views
- Cart additions
- Checkout events
- Payment events
- Search activity

This allows events to be processed independently and at high speed.

Customer Profiling

A customer profile database may store information such as purchase history, preferences, recently viewed products, and customer segments.

A distributed NoSQL database such as **Cassandra, MongoDB, or DynamoDB** could be used because these systems provide fast access and can scale horizontally.

Session Management

A caching system such as **Redis** could store active customer sessions.

For example, when a customer views several products, their recent activity can be stored in Redis. The recommendation system can access this information immediately without querying a large historical database.

Recommendation Workflow

The probable workflow is:

Customer Activity → Kafka → Stream Processing → Customer Profile → Recommendation Model → Personalized Product

Machine learning models could include:

- Collaborative filtering
- Content-based recommendation
- Matrix factorization
- Neural recommendation models

The model can use browsing behavior and previous purchases to recommend products.

Combining Structured and Semi-Structured Data

Order and payment information is structured and can be stored in relational databases or warehouses.

Browsing events are usually semi-structured JSON records. These can be stored in a data lake.

A distributed query engine such as **Presto/Trino or Spark SQL** can combine both datasets for analysis.

For example, customer browsing history can be joined with completed orders to determine which browsing patterns usually lead to purchases.

Fault Tolerance and Scalability

Kafka replication, distributed databases, and replicated storage can prevent data loss.

The platform can scale horizontally by adding more Kafka brokers, processing nodes, or database nodes.

Services can also be deployed across multiple availability zones to reduce the impact of hardware failures.

KPI Monitoring

Processed information can be sent to a data warehouse and connected to **Power BI, Tableau, or Looker**.

Important KPIs could include:

- Conversion rate
- Cart abandonment rate
- Average order value
- Product views
- Sales by region
- Recommendation click-through rate

Probable Architecture

Customer Activity
↓
Kafka
↓
Flink / Spark Streaming
↓
Redis + Customer Profile Database
↓
Recommendation Model
↓
Personalized Website

Historical data can simultaneously flow into:

Data Lake → Spark/Trino → Data Warehouse → BI Dashboard

3. Healthcare Big Data System

This system handles patient histories, medical images, wearable sensor information, and prescriptions. Since these datasets have very different formats, a hybrid storage architecture is likely required.

Hybrid Database Architecture

Structured information such as patient records, prescriptions, laboratory results, and appointments can be stored in relational databases such as **PostgreSQL or MySQL**.

Medical images such as X-rays, CT scans, and MRI scans are large unstructured files. They can be stored in object storage or a medical imaging system.

Wearable sensor data can be stored using time-series or NoSQL databases because the information arrives continuously.

Data Integration

Healthcare organizations use different systems, so interoperability is important.

Standards such as **HL7 and FHIR** can be used to exchange patient information between hospitals, laboratories, clinics, and insurance systems.

Integration tools such as **Apache NiFi, Kafka, or healthcare integration engines** can transfer and transform the data.

Streaming and Monitoring

Wearable devices continuously generate information such as heart rate, oxygen levels, and activity.

Kafka can receive this information, while **Apache Flink or Spark Streaming** can process it.

If an abnormal pattern is detected, the system can generate an alert for healthcare professionals.

Machine Learning

Machine learning can be applied to:

- Disease risk prediction
- Patient deterioration detection
- Medical image analysis
- Readmission prediction
- Treatment outcome prediction

Models such as Random Forest, XGBoost, neural networks, and deep learning models can be selected depending on the type of data.

Privacy and Security

Healthcare information is highly sensitive. The system would likely use:

- Encryption at rest
- Encryption during transmission
- Role-Based Access Control
- Multi-factor authentication
- Data masking
- Audit logging
- Patient data anonymization

Access should be limited according to the user's role.

Clinical Analytics Pipeline

Patient Records + Images + Wearables + Prescriptions

↓

Data Integration

↓

Secure Data Lake / Databases

↓

Spark / ML Processing

↓

Clinical Analytics

↓

Doctor Dashboard / Population Health Reports

Population-level analysis can help identify disease trends and high-risk groups while protecting individual patient identities.

4. Smart City Big Data Platform

The smart city platform receives continuous information from traffic sensors, CCTV systems, weather stations, and public transport systems. Because much of this information is time-sensitive, edge and stream processing are important.

Edge Computing

Processing some information close to the source reduces network traffic and latency.

For example, cameras can perform initial processing at the edge and send only metadata instead of continuously sending large video files to the central server.

Edge devices can identify:

- Vehicle counts
- Traffic density
- Accidents
- Parking availability

Messaging System

A distributed messaging system such as **Apache Kafka** can collect events from thousands of sensors.

Kafka topics could separate traffic, weather, CCTV metadata, and public transport information.

Centralized Storage

Historical data can be stored in a cloud data lake or HDFS.

Different databases may be used according to the data type:

- **TimescaleDB or InfluxDB** for time-series sensor data
- **PostgreSQL/PostGIS** for geographic data
- **NoSQL databases** for high-volume event data
- Data warehouses for reporting

Geospatial Analytics

Location information is important in smart city applications.

PostGIS can perform location-based queries such as identifying traffic congestion near a particular road or finding nearby public transport vehicles.

Real-Time Congestion Prediction

A streaming system such as **Apache Flink or Spark Streaming** can combine traffic and weather information.

For example:

Traffic Sensor Data + Weather Data + Transport Data
↓
Stream Processing
↓
Congestion Prediction Model
↓
Traffic Control / Alert

Machine learning models can predict traffic congestion based on historical traffic patterns, weather, time, and road conditions.

Batch and Real-Time Analytics

Real-time analytics can help control current traffic situations.

Batch analytics can study historical information for:

- Road planning
- Public transport planning
- Accident analysis
- Infrastructure development

Security

The system would likely use:

- Encryption
- Authentication
- Role-Based Access Control
- Network segmentation
- Secure APIs
- Audit logs
- Access monitoring

Sensitive infrastructure information should only be available to authorized personnel.

Visualization

Operational dashboards can be created using tools such as **Grafana, Power BI, or Tableau**.

Live dashboards may show:

- Traffic congestion
- Public transport locations
- Weather conditions
- Road incidents
- Sensor status

Probable Architecture

Sensors / CCTV / Transport / Weather
↓
Edge Processing
↓
Kafka
↓
Flink / Spark Streaming
↓
Time-Series DB + PostGIS + Data Lake
↓
ML Prediction + Historical Analytics
↓
Live City Dashboard

5. Healthcare Analytics and Predictive Health System

This system combines patient records, wearable data, laboratory reports, and appointment histories. Since the information includes both structured and unstructured medical data, a hybrid storage architecture is likely.

Hybrid Storage

Structured information such as patient details, appointments, prescriptions, and laboratory results can be stored in relational databases.

Wearable readings can be stored in time-series or NoSQL databases.

Medical documents and other unstructured files can be stored in a secure data lake or object storage.

ETL and Data Integration

Healthcare organizations may use different systems, so an ETL pipeline is needed.

The pipeline can perform:

Extract → Clean → Transform → Validate → Load

Standards such as **HL7 and FHIR** can help hospitals and clinics exchange healthcare information consistently.

Real-Time Monitoring

Wearable devices can continuously send patient information.

For example:

Wearable Device → Kafka → Stream Processing → Health Monitoring Model → Alert

If the patient's heart rate or another measurement becomes abnormal, the system can generate an alert.

Predictive Health Analytics

Machine learning can use historical patient information to calculate risk scores.

Possible applications include:

- Disease risk prediction
- Readmission prediction
- Patient deterioration prediction
- Treatment outcome prediction
- Early warning systems

Models such as **Logistic Regression, Random Forest, XGBoost, and neural networks** can be used.

Distributed Processing

Healthcare organizations may have millions of patient records. Processing all this information on one server would be inefficient.

Apache Spark can divide large datasets across multiple machines and process them in parallel.

This is particularly useful for population-level studies, such as identifying disease trends across different regions or age groups.

Data Governance and Compliance

Strong controls would be required because medical information is sensitive.

The platform would likely implement:

- Encryption at rest and in transit
- Role-Based Access Control
- Multi-factor authentication
- Data anonymization
- Audit trails
- Access monitoring
- Secure backups

- Data retention policies

Every access to sensitive information should be logged so that unauthorized activity can be investigated.

Diagnosis and Treatment Support

Machine learning models can provide risk scores to support doctors.

For example:

Patient Data + Lab Results + Wearable Data + Medical History
↓
Feature Engineering
↓
ML Model
↓
Risk Score
↓
Clinical Decision Support

The system should support healthcare professionals rather than independently making medical decisions.

Probable Architecture

Hospital / Clinic / Wearable / Laboratory Data
↓
FHIR/HL7 + ETL Pipeline
↓
Secure Data Lake + Relational DB + NoSQL/Time-Series DB
↓
Kafka + Spark/Flink
↓
Machine Learning Models
↓
Risk Scores & Analytics
↓
Clinical Dashboard

Overall Conclusion

The five systems show how large-scale applications are generally built using a combination of **distributed storage, data ingestion, stream processing, batch processing, machine learning, security, and visualization**.

Although the exact technologies cannot be known from the descriptions alone, systems such as **Kafka, Spark, Flink, Hadoop, NoSQL databases, cloud data lakes, Redis, PostGIS, and BI tools** are probable choices because they address the scalability, real-time processing, storage, and analytics requirements of these applications.

The main design principle across all five cases is to separate **data collection, storage, processing, analytics, and presentation layers**. This makes the system easier to scale, maintain, secure, and adapt as data volumes increase.